

### ### Addition & Subtraction for Signed Binary Integers (Using Sign and Magnitude)

In sign and magnitude representation, the first bit (most significant bit) is the sign bit: 0 means positive, 1 means negative. The rest of the bits represent the magnitude (absolute value) of the number. For example, in 4 bits, +3 is 0011, and -3 is 1011.

Subtraction is turned into addition by using the negative of the second number:  $X - Y = X + (-Y)$ . The same for other combinations like  $-X - Y = (-X) + (-Y)$ .

#### \*\*Rules:\*\*

1. Look at the sign bits (but ignore them for the actual math on magnitudes).
2. If signs are the same:
  - Add the magnitudes.
  - Give the result the same sign as the originals.
3. If signs are different:
  - Compare the magnitudes.
  - Subtract the smaller magnitude from the larger one.
  - Give the result the sign of the larger magnitude.
4. Watch for overflow: If adding magnitudes causes a carry into the sign bit, it might flip the sign wrongly.

#### \*\*Harder Example: +13 + (-5) in 6 bits (result should be +8).\*\*

- +13 in 6 bits: 0 1101 (wait, 13 is 001101, so 0 01101).
- -5 in 6 bits: 1 00101 (magnitude of 5 is 000101).
- Signs different: 0 and 1.
- Compare magnitudes: 01101 (13) > 00101 (5).
- Subtract smaller from larger: 01101 - 00101.
- Binary subtraction: Start from right.
  - 1 - 1 = 0.
  - 0 - 0 = 0.
  - 1 - 1 = 0.
  - 1 - 0 = 1.
  - 0 - 0 = 0.
- Result: 01000 (8).
- Sign of larger (which was +13, sign 0): So +8 is 0 01000.
- Full step-by-step: We treated it as 13 + (-5), but since signs differ, subtract magnitudes and take positive sign.

#### \*\*Another Harder Example: -12 - 7 in 6 bits (result -19, but check overflow).\*\*

- -12: 1 01100 (12 is 001100).
- -7: 1 00111 (7 is 000111).
- Signs same (both 1, negative).
- Add magnitudes: 01100 + 00111 = 10011 (19).
- Sign: 1 (negative), so 1 10011 = -19.
- In 6 bits, magnitude is 5 bits, sum 10011 fits, no overflow.
- But if sum was bigger, like -15 - 1 = -16: Magnitudes 01111 + 00001 = 10000 (overflow into 6th bit? Magnitude bits are 5, sum starts with 1, which might be seen as overflow if not handled).

Note: Overflow example from doc: 0101 + 0110 = 1011, which is 5+6=11, but in sign-magnitude it looks like -3 wrongly due to carry affecting sign.

### ### Multiplication and Division for Unsigned Binary Integers

Unsigned means all bits are positive value, no sign bit.

#### \*\*Multiplication Rules:\*\*

- Find bits needed: Enough to hold the product (up to 2n bits for n-bit numbers).

- Use binary version of long multiplication: Shift and add partial products.
- Trim extra leading zeros.

**\*\*Harder Example:  $13 * 5$  in 6 bits (result 65). \*\***

- 13: 001101, 5: 000101. Use 6 bits.
- Long multiplication:
  - Multiply by least bit of 5 (1):  $001101 * 1 = 001101$ .
  - Shift left 1 (next bit 0):  $001101 \ll 1 = 011010$ ,  $*0 = 000000$ .
  - Shift left 2 (next 1):  $011010 \ll 1 = 110100$ ,  $*1 = 110100$ . Wait, no: Actually, partials based on multiplier.
  - Better: Multiplier 000101 (5), multiplicand 001101 (13).
  - Bit 0 (1): Add 001101 shifted 0: 001101.
  - Bit 1 (0): Add 0 shifted 1.
  - Bit 2 (1): Add 001101 shifted 2: 00110100 (but in bits:  $001101 \ll 2 = 0110100$ , wait for 7-8 bits).
  - Total:  $001101 + 0110100 = 10000101$  (but align properly).
  - Step: Start with 0.
  - Rightmost bit of 5 is 1: Add 001101.
  - Shift product right or multiplier, but standard:  $13 * 5 = 65 = 1000001$  in binary.
  - Binary: 1000001 is  $64 + 1 = 65$ .
- Detail: Use loop: For each 1 in multiplier, add multiplicand shifted by position.

**\*\*Division Rules:\*\***

- Dividend (top), divisor (bottom).
- Take chunks of dividend same size as divisor.
- If chunk  $\geq$  divisor, subtract and put 1 in quotient; else 0, bring down next bit.
- Continue until done, remainder is left.

**\*\*Harder Example:  $65 / 5$  (result 13, remainder 0). \*\***

- 65: 1000001, 5: 101. Use 7 bits for 65.
- Start: Take first 3 bits (size of divisor): 100 (4) < 101 (5)? No, quotient bit 0, bring down next: 1000 (8) > 5? Yes, subtract 101 from 1000:  $1000 - 101 = 011$  (3), wait binary sub.
- Actually: Long division.
- 1000001 divided by 101.
- How many times 101 into 100 (4)? 0, bring 0: 1000 (8) > 5,  $101 * 1 = 101$ ,  $1000 - 101 = 011$  (3), bring 0: 0110 (6) > 5? 1,  $6 - 5 = 1$ , bring 0: 0100 (4) < 5? 0, bring 1: 01001 (9) > 5? 1,  $9 - 5 = 4$ , but wait for full.
- Quotient: 1101 (13), remainder 0? Wait,  $5 * 13 = 65$ , yes.
- Step: Align, subtract repeatedly.

**\*\*Example with Remainder:  $67 / 5 = 13$  remainder 2. \*\***

- 67: 1000011, 5: 101.
- Similar, but end with remainder 010 (2).

### ### Multiplication and Division for Signed Binary Integers (Using Sign and Magnitude)

**\*\*Rules:\*\***

- Ignore signs for the math.
- Do unsigned multiplication or division on magnitudes.
- For sign of result:
  - Multiplication/Division: If signs same, result positive (sign 0); different, negative (sign 1).

**\*\*Harder Multiplication Example:  $-13 * 5 = -65$  in 8 bits. \*\***

- Magnitudes: 13 (00001101), 5 (00000101).
- Multiply as unsigned:  $13 * 5 = 65 = 01000001$ .
- Signs different (1 and 0), so result sign 1: 1 01000001 (but pad to 8 bits: sign + 7 magnitude bits? Assume 8 total, sign + 7).
- Detail: Multiply magnitudes as above, then apply sign.

**\*\*Harder Division Example:  $65 / -5 = -13$  in 8 bits.\*\***

- Magnitudes:  $65 / 5 = 13$ .
- Signs different (0 and 1), result sign 1: -13.

**\*\*With Remainder:  $-67 / 5 = -13$  remainder -2? \*\*** But signs affect remainder too, but doc focuses on quotient sign.

### ### Binary Complements

Complements help represent negatives without separate sign bit, making math easier (subtraction = addition).

#### #### 1's Complement

- Flip all bits: 0 to 1, 1 to 0.
- Positive numbers unchanged, negatives are flip of positive.

**\*\*Harder Example: 1's complement of +13 in 8 bits.\*\***

- +13: 00001101.
- Flip: 11110010 = -13 in 1's complement.

Problem: Two zeros (+0=00000000, -0=11111111), and some math issues.

#### #### Subtraction Using 1's Complement

- $X - Y = X + (1's \text{ complement of } Y)$ .
- If carry out (end-around carry), add it back to result.

**\*\*Harder Example:  $13 - 5 = 8$  in 8 bits.\*\***

- 13: 00001101.
- 1's comp of 5: Flip 00000101 = 11111010.
- Add: 00001101 + 11111010 = 100000111 (carry 1).
- Add carry back: 00000111 + 1 = 00001000 (8).
- Detail: Bit by bit add, carries propagate.

**\*\*Problem Example:  $-2 - (-2) = 0$ , but in 4 bits:  $-2 = 1101$  (flip 0010), add  $1101 + 1101 = 11010$ , carry 1, add back  $1010 + 1 = 1011 = -4$  wrongly? Doc mentions issues like this.\*\***

#### #### 2's Complement

- 1's complement +1.
- Fixes zero issue: Only one zero, negatives start with 1.

**\*\*Harder Example: 2's complement of -13 in 8 bits.\*\***

- Positive 13: 00001101.
- 1's: 11110010.
- +1: 11110011 = -13.

#### #### Subtraction Using 2's Complement

- $X - Y = X + (2's \text{ complement of } Y)$ .
- Ignore (drop) any final carry.

**\*\*Harder Example:  $13 - 5 = 8$  in 8 bits.\*\***

- 13: 00001101.
- 2's of 5: Flip 00000101 = 11111010 + 1 = 11111011.
- Add: 00001101 + 11111011 = 100000000 (carry 1, drop it): 00001000 (8).

**\*\*Why Better: For  $-2 - 2$  in 4 bits:  $-2 = 1110$  (2's of 2 = 0010 flip 1101 + 1 = 1110). Add  $1110 + 1110 = 11100$ , drop carry 1:  $1100 = -4$ ? Wait,  $-2 - 2 = -4$  yes. But earlier 1's issue fixed because no end-around weirdness.\*\***

### ### Multiplication Using 1's Complement

Use regular multiplication, but since negatives are 1's flips, result might need adjustment.

**\*\*Harder Example:  $5 * -3$  in 6 bits.\*\***

- 5: 000101, -3: 1's of 3=111100 (flip 000011=111100).
- Multiply as is, but doc shows issue: Like  $2 * -2 = 0010 * 1101$ , product 1010, which is -5 but should be -4.
- Problem: Overflow or representation error, solved by sign extension or Booth's algorithm, but 1's has flaws.

**\*\*Why Happening: 1's complement has asymmetry around zero, leading to off-by-one in some multiplies. Solved by using 2's instead.\*\***

### ### Multiplication Using 2's Complement

Treat as unsigned multiplication, but interpret as signed. Works directly because 2's is weighted properly.

**\*\*Harder Example:  $5 * -3 = -15$  in 6 bits.\*\***

- 5: 000101, -3: 2's=111101 (flip 000011=111100+1=111101).
- Multiply: Like unsigned, partial adds.
  - Result: 11110001, but in 8 bits?  $5 * -3 = -15 = 11110001$  in 8-bit 2's.
- Detail: Bit shifts and adds give correct signed result.

### ### Division Using 2's Complement

Convert to subtractions: Count how many subtractions to reach zero.

**\*\*Sign Rules:\*\***

- If dividend/divisor signs differ, quotient negative.
- Use absolute, but in 2's, do signed subs.

**\*\*Harder Example:  $25 / -5 = -5$  in 6 bits.\*\***

- 25: 011001, -5: 111011 (2's).
- But division as repeated sub: Start with positive? Doc: Translate to subtractions.
- $25 - (-5) = 25 + 5$ , but for quotient.
- Doc example:  $16 / -4 = -4$ , by repeated add of 2's comp of -4 (which is positive 4's 2's? Wait).
- Steps: For negative result, final quotient in 2's negative.
- With remainder:  $7 / -3$  in 4 bits.
  - 7: 0111, -3: 1101 (2's of 3=0011 flip 1100+1=1101).
  - Division: Quotient -2 (since  $3 * 2 = 6$ ,  $7 - 6 = 1$ , but sign negative).
  - Repeated: But doc discussion: For remainder, remainder same sign as dividend.

### ### Floating-Point Number Representation

Floating-point represents real numbers with fraction (mantissa) and exponent, like scientific notation.

Binary:  $F * 2^E$ , where F is normalized (1.something).

Not using complements directly; complements are for integers. (Query says "both compliments" maybe typo for "using complements"? But it's separate.)

Loss of precision: Fixed bits can't represent all reals exactly.

**\*\*Example: 5.75 in binary float.\*\***

- $5 = 101$ ,  $0.75 = 0.11$ , so  $101.11 = 1.0111 * 2^2$ .

### ### IEEE-754 32-bit Single-Precision Floating-Point Numbers

32 bits: 1 sign, 8 exponent (biased +127), 23 fraction (implicit 1.).

**\*\*Normalized Form:\*\***

- Fraction: 1.F (implicit 1).
- Exponent: Actual E +127 (bias for negative exponents).

**\*\*Harder Example: Represent -10.625 in IEEE-754.\*\***

- -10.625: Binary  $1010.101 = 1.010101 \cdot 2^3$ .
- Sign: 1.
- Fraction: 010101000... (23 bits, pad zeros).
- Exponent:  $3+127=130=10000010$ .
- Full: 1 10000010 0101010000000000000000.

**\*\*Decode Example from Doc:  $1\ 10000001\ 011000000000000000000000 = -1.375 \cdot 2^{(129-127)} = -1.375 \cdot 4 = -5.5$ .\*\***

Here is a clearer, simpler explanation of the key parts from **\*\*Chapter 5: Computer Arithmetic\*\*** — using very easy English + harder (more realistic) examples so you can really understand how it works in practice.

I will go section by section, explain the idea simply, show the rule, then give a harder example with full step-by-step calculation.

### ### 1. Addition & Subtraction – Sign and Magnitude (Signed Binary)

**\*\*Simple idea\*\***

- First bit = sign (0 = positive, 1 = negative)
- Rest of bits = size (magnitude)
- To subtract → change to addition of negative:  $X - Y = X + (-Y)$
- Two cases:

**\*\*Case A: Same sign\*\*** → just add the magnitudes → keep the same sign

**\*\*Case B: Different signs\*\*** → subtract smaller magnitude from larger → take sign of the larger one

**\*\*Harder example – same sign\*\***

Calculate:  $-19 + (-13)$  using 8 bits

$-19 \rightarrow$  sign 1, magnitude 19 = 10011  $\rightarrow$  1 00010011

$-13 \rightarrow$  sign 1, magnitude 13 = 01101  $\rightarrow$  1 00001101

Magnitudes: 00010011 + 00001101

...

00010011  
+ 00001101  
-----

00100000 = 32 (but we only have 7 bits for magnitude → overflow!)

...

Problem: overflow happened → in sign-magnitude this is dangerous because carry can go into sign bit and ruin the sign.

Correct answer should be  $-32$ , but many simple sign-magnitude systems can't represent  $-32$  properly in 8 bits.

**\*\*Harder example – different signs\*\***

Calculate:  $25 - 37$  (should be  $-12$ ) using 8 bits

25 → 0 00011001  
37 → 1 00100101

Different signs → compare magnitudes: 25 < 37  
Subtract smaller from larger: 00100101 – 00011001  
...

```
  00100101
- 00011001
-----
  00001100 = 12
...
```

Take sign of larger number (which was negative) → -12 → 1 00001100

### ### 2. Multiplication & Division – Unsigned Binary

#### \*\*Multiplication (unsigned)\*\*

Like normal long multiplication but in binary — shift and add only when bit is 1.

#### \*\*Harder example\*\*

13 × 11 = 143 (using enough bits — 8-bit result)

13 = 00001101

11 = 00001011

...

```
  00001101 ← 13
× 00001011 ← 11
-----
```

00001101 ← ×1 (bit 0)

00000000 ← ×1 shifted 1 (but bit 1=1? wait no bit1=1 → wait)

Actually step by step:

```
Bit 0 (1):  00001101
Bit 1 (1):  + 00011010 (13 << 1)
Bit 2 (0):  + 00000000
Bit 3 (1):  + 001101000 (13 << 3)
-----
Total:      10001111 = 143 (correct)
...
```

#### \*\*Division (unsigned) – restoring method\*\*

Harder example: 113 ÷ 7 (should be 16 remainder 1)

113 = 1110001

7 = 111

Long division steps (like decimal but binary):

- 111 into 111 → 1, subtract → 000, bring down 0 → 000
- 111 into 000 → 0, bring down 0 → 000
- 111 into 000 → 0, bring down 1 → 001
- 111 into 001 → 0, bring down nothing → remainder 1

Quotient = 10000 (16), remainder = 1

113 = 7×16 + 1 → correct

### ### 3. Multiplication & Division – Signed (Sign & Magnitude)

**\*\*Rule\*\***

- Ignore signs → do unsigned multiplication / division on magnitudes
- Sign of result = XOR of the two signs  
(same signs → + different signs → -)

**\*\*Harder multiplication example\*\***

$$-29 \times 14 = -406$$

Magnitudes:  $29 \times 14$

$$29 = 11101$$

$$14 = 01110$$

Result magnitude = 101100110 (406)

Signs different → sign bit = 1

Final → 1 101100110 (-406)

**### 4. 1's Complement & Problems**

**\*\*1's complement\*\*** = flip every bit

**\*\*Subtraction\*\*** = add 1's complement + end-around carry (add carry-out back to LSB)

**\*\*Big problem example\*\***  $-2 - 2$  using 4 bits

$$+2 = 0010$$

$$-2 \text{ in } 1's = 1101$$

$$1101 + 1101 = 1\ 1010$$

End-around carry:  $1010 + 1 = 1011 = -4$  in 1's complement

But  $-2 - 2 = -4 \rightarrow$  correct here, but...

**\*\*Real serious problem\*\***

$$2 \times -2 \text{ should be } -4$$

But in 1's complement multiplication often gives wrong answer (off by 1) because  $-0$  exists and range is asymmetric.

That's why almost no one uses 1's complement today.

**### 5. 2's Complement – The Standard Today**

**\*\*2's complement\*\*** = 1's complement + 1

**\*\*Subtraction\*\*** = add 2's complement → ignore (drop) final carry

**\*\*Hard example\*\***

$$-47 + 19 \text{ using 8 bits (should be } -28)$$

$$+19 = 00010011$$

$$-47 \text{ in } 2's = \text{invert } 00101111 \rightarrow 11010000 + 1 = 11010001$$

Add:

``

$$\begin{array}{r} 00010011 \\ + 11010001 \\ \hline \end{array}$$

11100100 (carry out dropped)

``

11100100 in 2's complement =  $-28 \rightarrow$  correct

**### 6. Multiplication in 2's Complement**

Just multiply as if unsigned → result is correct in 2's complement (as long as you keep enough bits)

**\*\*Hard example\*\***

$-19 \times 13 = -247$  using 9–10 bits

$-19$  in 8-bit 2's = 11101101

$13 = 00001101$

Multiply normally (Booth's algorithm makes it faster in hardware, but basic method works).

Result in 2's = 111100001001 → correct  $-247$

### ### 7. Division in 2's Complement

Usually done by:

- Take absolute values
- Do unsigned division
- Apply correct sign to quotient
- Remainder usually has same sign as dividend

**\*\*Hard example with remainder\*\***

$7 \div -3$  (should be quotient  $-2$ , remainder 1)

$7 = 0111$

$-3$  in 2's (4 bits) = 1101

Unsigned magnitudes:  $7 \div 3 = 2$  remainder 1

Signs different → quotient negative →  $-2$

Remainder positive (same as dividend) →  $+1$

Check:  $-3 \times -2 = 6$ ,  $7 - 6 = +1$  → correct

### ### 8. Floating-Point (IEEE 754 32-bit Single Precision)

**\*\*Simple idea\*\***

Sign (1 bit) + Exponent (8 bits, biased by 127) + Fraction (23 bits)

Real value =  $(-1)^{\text{sign}} \times (1.\text{fraction}) \times 2^{(\text{exponent} - 127)}$

**\*\*Hard example – convert  $-12.625$  to IEEE 754\*\***

1.  $-12.625$

2. Positive part: 12.625

3. Binary: 1100.101

4. Normalize:  $1.100101 \times 2^3$

5. Fraction: 10010100000000000000000 (23 bits)

6. Exponent:  $3 + 127 = 130 = 10000010$

7. Sign: 1 (negative)

Final 32 bits:

1 10000010 100101000000000000000000

=  $-12.625$  exactly

Another example from your notes:

1 10000001 011000000000000000000000

→ sign=1, exp= $129 - 127 = 2$ , fraction= $1.011 = 1.375$

→  $-1.375 \times 2^2 = -5.5$

Let me know which part you want even more examples or practice questions for!